PRACTICAL IMPLEMENTATION GUIDE

Install Terraform on RHEL and Provision an F5 BIG-IP VIP

A step-by-step lab guide with configuration examples, validation commands and troubleshooting

Platform	Red Hat Enterprise Linux 8 / 9
Infrastructure	F5 BIG-IP – Local Traffic Manager (LTM)
Automation	Terraform CLI + F5Networks BIG-IP Provider
Prepared by	Kailas Pathade

Version: 1.0 | Prepared: November 2026

1. Purpose and scope

This guide explains how to install Terraform on a Red Hat Enterprise Linux (RHEL) host, configure the F5 BIG-IP Terraform provider, and deploy a sample HTTPS virtual server (VIP) with an HTTP health monitor, a load-balancing pool and two pool members.

What the lab creates

- HTTP health monitor: /Common/http_monitor
- Pool: /Common/web_pool using round-robin load balancing
- Pool members: 10.10.20.11:80 and 10.10.20.12:80
- Virtual server: /Common/terraform_https_vip at 10.10.10.100:443
- SNAT: Automap; virtual server protocol: TCP

Important: This is a lab example. Replace all sample IP addresses, credentials and object names with values approved for your environment. Confirm that the VIP address is available and routed before deployment.

2. Prerequisites

- A RHEL 8 or RHEL 9 system with sudo/root access and outbound access to the HashiCorp RPM repository.
- A reachable F5 BIG-IP management address with the iControl REST API enabled and HTTPS management access (normally TCP/443).
- A BIG-IP account authorized to create and manage the required LTM objects in the target partition.
- Network reachability from the RHEL host to the BIG-IP management interface.
- Backend servers reachable from the BIG-IP on TCP/80, with an HTTP service that can respond to the configured monitor.
- A planned, unused VIP address and an approved change window for shared or production environments.

Lab values used in this example

Parameter	Example value	Purpose
RHEL host	RHEL 8 / 9	Terraform execution host
F5 management IP	192.168.1.129	BIG-IP API endpoint
VIP address	10.10.10.100	Client-facing virtual address
VIP port	443	HTTPS listener port
Pool member 1	10.10.20.11:80	Backend web server
Pool member 2	10.10.20.12:80	Backend web server
Partition	/Common	F5 configuration partition

3. Install Terraform on RHEL

Step 1 – Confirm RHEL version and architecture

```
cat /etc/redhat-release
uname -m
```

Confirm that the host is running a supported RHEL release and that the architecture is appropriate for the Terraform package (commonly x86_64).

Step 2 – Install repository management utilities

```
sudo dnf install -y yum-utils
```

Step 3 – Add the HashiCorp RPM repository

```
sudo yum-config-manager --add-repo https://rpm.releases.hashicorp.com/RHEL/hashicorp.repo
```

Step 4 – Install Terraform

```
sudo dnf install -y terraform
```

Step 5 – Verify the installation

```
terraform --version
which terraform
```

Expected binary location on a standard package installation: /usr/bin/terraform. The version output should report Terraform v1.x.x and the host platform.

Step 6 – Create the project directory

```
mkdir -p ~/f5-terraform
cd ~/f5-terraform
```

Tip: Run Terraform commands from the directory containing your .tf files. Terraform automatically loads all .tf files in the current working directory.

4. Create the Terraform project

Create the following four files in the same directory. The corrected filename is variables.tf (not varibles.tf).

```
f5-terraform/
├── provider.tf
├── variables.tf
├── main.tf
└── outputs.tf
```

4.1 variables.tf – Define input variables

```
variable "f5_address" {
  description = "F5 BIG-IP management IP or hostname"
  type        = string
}

variable "f5_username" {
  description = "F5 BIG-IP username"
  type        = string
  sensitive   = true
}

variable "f5_password" {
  description = "F5 BIG-IP password"
  type        = string
  sensitive   = true
}

variable "vip_address" {
  description = "Virtual server destination IP"
  type        = string
```

```
  default     = "10.10.10.100"
}

variable "vip_name" {
  description = "Virtual server name"
  type        = string
  default     = "terraform_https_vip"
}
```

4.2 provider.tf – Configure the F5 provider

```
terraform {
  required_providers {
    bigip = {
      source  = "F5Networks/bigip"
      version = "~> 1.22"
    }
  }
}

provider "bigip" {
  address  = var.f5_address
  username = var.f5_username
  password = var.f5_password
}
```

The provider communicates with BIG-IP through the iControl REST API. The address value should be the management IP/hostname only (for example, 192.168.1.129); do not paste a label or explanatory text into the value.

4.3 main.tf – Create monitor, pool, members and VIP

```
# HTTP health monitor
resource "bigip_ltm_monitor" "http_monitor" {
  name     = "/Common/http_monitor"
  parent   = "/Common/http"
  interval = 5
  timeout  = 16
}

# Backend pool
resource "bigip_ltm_pool" "web_pool" {
  name                = "/Common/web_pool"
  load_balancing_mode = "round-robin"
  monitors            = [bigip_ltm_monitor.http_monitor.name]
}

# Backend server 1
resource "bigip_ltm_pool_attachment" "server1" {
  pool       = bigip_ltm_pool.web_pool.name
  node       = "10.10.20.11:80"
  depends_on = [bigip_ltm_pool.web_pool]
}

# Backend server 2
resource "bigip_ltm_pool_attachment" "server2" {
  pool       = bigip_ltm_pool.web_pool.name
  node       = "10.10.20.12:80"
  depends_on = [bigip_ltm_pool.web_pool]
}

# HTTPS virtual server
```

```
resource "bigip_ltm_virtual_server" "https_vip" {
  name                        = "/Common/${var.vip_name}"
  destination                 = var.vip_address
  port                        = 443
  ip_protocol                 = "tcp"
  pool                        = bigip_ltm_pool.web_pool.name
  profiles                    = ["/Common/tcp"]
  source_address_translation = "automap"

  depends_on = [
    bigip_ltm_pool_attachment.server1,
    bigip_ltm_pool_attachment.server2
  ]
}
```

Note: This example attaches the standard /Common/tcp profile. It does not attach a Client SSL profile, so it provisions a TCP virtual server listening on port 443; add an approved Client SSL profile and any required HTTP profile if you need the BIG-IP to terminate/decrypt client-side HTTPS.

4.4 outputs.tf – Display deployment outputs

```
output "vip_name" {
  value = bigip_ltm_virtual_server.https_vip.name
}

output "vip_destination" {
  value = bigip_ltm_virtual_server.https_vip.destination
}

output "backend_pool" {
  value = bigip_ltm_virtual_server.https_vip.pool
}
```

5. Provide credentials safely

Do not hard-code the BIG-IP password in provider.tf or commit credentials to Git. For an interactive lab, set Terraform input variables as environment variables in the current shell:

```
export TF_VAR_f5_address="192.168.1.129"
export TF_VAR_f5_username="your_f5_username"
read -s -p "F5 password: " TF_VAR_f5_password
export TF_VAR_f5_password
echo
```

Terraform will read these environment variables automatically. The password is not echoed while typing. Clear the shell variable when the session is complete:

```
unset TF_VAR_f5_password
```

State-file warning: Terraform state can contain sensitive infrastructure details. Restrict access to terraform.tfstate, use an approved remote backend with encryption and access controls for team/production use, and never upload state files to public repositories.

6. Initialize, validate and plan

Step 1 – Initialize Terraform and download the provider

```
terraform init
```

Successful initialization confirms Terraform can resolve and install the configured provider. A .terraform directory and lock file are created.

Step 2 – Format and validate the configuration

```
terraform fmt -recursive
terraform validate
```

Expected validation result: Success! The configuration is valid.

Step 3 – Review the execution plan

```
terraform plan
```

Terraform reads the three TF_VAR_* values from the environment, refreshes managed objects and displays proposed changes. Review the plan carefully. In a clean environment where none of the five objects exists, expect five resources to be added: one monitor, one pool, two pool attachments and one virtual server. If an object (such as the monitor in the original lab run) already exists and is tracked or refreshed, the plan may show fewer additions.

If you prefer interactive prompts, run terraform plan without arguments and enter each requested variable value. Do not type descriptive labels such as “F5 management IP address” as part of a command or variable value.

Optional – Save and apply the reviewed plan

```
terraform plan -out=f5-vip.tfplan
terraform show f5-vip.tfplan
terraform apply f5-vip.tfplan
```

Using a saved plan helps ensure that the actions applied are the actions you reviewed. Protect the plan file as it may contain sensitive values.

7. Apply the configuration

If you are not using a saved plan, execute:

```
terraform apply
```

Review the proposed actions. Type yes only after confirming the target BIG-IP, partition, VIP address, pool members and change approval.

A successful run ends with an apply summary and output values similar to:

```
Apply complete! Resources: 4 added, 0 changed, 0 destroyed.

Outputs:
backend_pool     = "/Common/web_pool"
vip_destination  = "10.10.10.100"
vip_name         = "/Common/terraform_https_vip"
```

The resource count in the original lab output was four additions because the provider's resource output groups the monitor/pool/member/VIP operations as reported by that run. Always use the actual Terraform apply summary as the source of truth for your provider version.

8. Verify on BIG-IP and test traffic

Log in to the BIG-IP Configuration utility and verify the following under Local Traffic:

- Monitors: /Common/http_monitor exists and is configured with interval 5 and timeout 16.
- Pools: /Common/web_pool exists, uses Round Robin and references the HTTP monitor.
- Pool Members: 10.10.20.11:80 and 10.10.20.12:80 are present and enabled.
- Virtual Servers: /Common/terraform_https_vip listens on 10.10.10.100:443 and references /Common/web_pool.
- SNAT: Automap is configured on the virtual server.

From a client that has a route to the VIP, test the listener (replace the sample VIP if changed):

```
curl -vk https://10.10.10.100/
```

Because this example does not configure a Client SSL profile, a successful TLS handshake is not guaranteed. For an HTTPS termination test, configure and attach a suitable Client SSL profile and ensure the backend application and health monitor are appropriate.

9. Common issues and troubleshooting

Symptom	Recommended check
Terraform command reports missing variable values	Export TF_VAR_f5_address, TF_VAR_f5_username and TF_VAR_f5_password, or run terraform plan/apply and answer the prompts.
Invalid character or space in host name	Check that f5_address contains only the BIG-IP management IP/hostname. Do not include explanatory text, spaces, a prompt label or an unintended URL.
Connection timeout / connection refused	Verify routing, firewall rules, management HTTPS access on TCP/443, BIG-IP availability and that the RHEL host can reach the management interface.
Authentication or authorization failure	Verify username/password and ensure the account has permissions to create the required LTM objects.
Provider initialization error	Check repository access, DNS/proxy settings and the provider source/version in provider.tf; rerun terraform init after changing provider requirements.
Pool members show down	Check backend server availability, routing from BIG-IP, TCP/80 access, HTTP response behavior and monitor configuration.
Terraform detects out-of-band changes	Someone or another automation tool changed the BIG-IP outside Terraform. Review the refreshed plan before applying; do not blindly overwrite changes.
VIP already exists	Confirm ownership and intended configuration. Import an existing object into state or choose a different approved name/address; do not create duplicates.

10. Terraform commands

```
terraform state list        # List resources tracked in state
terraform show              # Display current state details
terraform plan              # Preview drift and proposed changes
terraform apply             # Apply reviewed changes
terraform destroy           # Destroy resources managed by this configuration
```

Destructive operation: terraform destroy removes resources managed by this Terraform state. Use only in a disposable lab after verifying the target and obtaining approval. It is not a routine cleanup command for shared or production BIG-IP systems.

11. Operational recommendations

- Use separate Terraform workspaces/state and credentials for development, test and production.
- Keep Terraform configuration in version control, but exclude passwords, *.tfvars containing secrets, state files, plan files and local provider directories.

- Use a dedicated least-privilege BIG-IP automation account and rotate credentials according to policy.
- Back up BIG-IP configuration and confirm HA/synchronization status before significant changes.
- Review terraform plan output and peer-review changes before apply in controlled environments.
- Pin and test provider versions; upgrade deliberately after checking provider release notes and resource compatibility.
- Use a secure remote state backend with locking, encryption, backups and role-based access for team workflows.

12. Reference links

- **Terraform installation on RHEL – HashiCorp RPM repository**
  https://rpm.releases.hashicorp.com/
- **F5 BIG-IP Terraform Provider – Terraform Registry (version 1.22.0)**
  https://registry.terraform.io/providers/F5Networks/bigip/1.22.0/docs
- **F5 BIG-IP Terraform Provider – latest documentation**
  https://registry.terraform.io/providers/F5Networks/bigip/latest/docs
- **F5 Resources for Terraform – F5 documentation**
  https://clouddocs.f5.com/products/orchestration/terraform/latest/

SUPPORT & CONTACT

Kailas Pathade

F5 BIG-IP | Load Balancing | Network Automation

Name	**Kailas Pathade**
Email	**kailasdreams@gmail.com**
Mobile	**+91 7777050805**

For technical guidance, implementation support or training enquiries, please contact me.